新员工节点注册方案

AutoMage-2

新员工节点注册方案设计

版本：v2.0（验证版）　日期：2026-05-06 {align="center"}

机密文档 · 仅供内部讨论

一、项目背景

AutoMage-2 采用「多级节点」架构，核心设计理念是为每个岗位配备「数字分身」Agent，通过标准化 Schema 实现数据契约化传输，采用分层数据汇总机制降低核心层负载。

根据里程碑规划，系统需要支持企业内新员工的快速接入：新员工注册后，系统自动为其创建对应的 Agent 节点实例，使其能够立即参与企业的数字化协作流程（日报填写、任务接收、决策参与等）。

然而，新员工注册流程涉及多个系统的协同：前端注册页面、后端数据库、Agent 实例管理、飞书消息路由、Gateway 通信等。如何设计一个简洁、可扩展的注册流程，是本方案的核心问题。

二、问题分析

2.1 注册时机问题

问题：新员工注册时机是「先下发实例再走数据库 Skill 写入」，还是「直接一个前端页走企业内注册流程 + 后端调数据库 API 写入」？

分析： Skill 是 AutoMage-2 中 Agent 读写数据库的标准接口（里程碑二的核心链路：数据库 → CRUD API → 数据库 Skill → Agent 联调）。但新员工注册是「人」的操作，不是 Agent 的操作。如果让 Skill 参与注册流程，会增加不必要的抽象层，且违背了 Skill 的设计初衷——Skill 是给 Agent 日常读写用的，不是给人注册用的。

结论： 注册走前端页面 + 后端 API 直接写入数据库，Agent 实例注册完成后再加载 Skill。

2.2 Gateway 接入问题（核心难点）

建实例本身很简单——对主机来说就是多开一个 Docker 容器。真正的难点在于 Gateway 的接入：如何让新创建的 Agent 实例能够收发消息？

方案对比：

方案	优点	缺点	推荐度
独立应用注册（自动创建）	每个实例独立飞书应用，天然隔离；Gateway 路由简化为 bot_id → tenant_id	需飞书企业免审开发权限；每人一次扫码（~30s）	✓（推荐）
共享应用 + 多租户路由	一次配置，全员共享；Gateway 按 user_id 路由，隔离性好	需开发 Gateway 路由层；共享 App 挂掉全员失联	✓（兼容方案）
共享应用 + 群聊模式	实现简单	消息隔离差，无法支持分层决策，违背多级节点架构	✗

核心矛盾（已解决）： 原方案中"员工需手动注册应用、配置权限、获取 app_id/app_secret，体验极差"的问题，已通过 **飞书企业免审开发权限 + lark-cli 自动创建模板 Bot** 解决。新员工只需扫码或点击链接（~30s），PersonalAgent 框架自动配置 bot 能力、WebSocket 事件订阅、32 个权限 scope，零手动控制台操作。实证验证：app_id cli_a971b4bfbdbadcce，全流程 ~2 分钟，飞书企业无应用数量限制。

2.3 权限隔离问题

共享飞书应用不代表数据可以混在一起。不同员工、不同层级（Staff/Manager/Executive）的数据必须严格隔离，包括：

- 数据库读写隔离：每个 Agent 只能访问自己 tenant_id 下的数据
- 消息路由隔离：飞书消息必须准确路由到对应的 Agent 实例
- API 鉴权隔离：调用数据库 API 时需要验证身份和权限

三、解决方案：独立 Bot 自动创建 + Gateway 路由

3.1 架构设计

每个员工拥有独立的飞书 Bot（通过免审权限 + lark-cli 自动创建），通过 bot_id 做路由标识。飞书消息从各 Bot 的 Webhook 到达 Gateway，由 Gateway 根据 bot_id → employee_id → tenant_id 路由到对应的 Agent 实例。

架构示意：

飞书消息 → 员工专属 Bot（每人一个，自动创建）

↓

Gateway 路由层根据 bot_id → employee_id → tenant_id

↓

┌──────┬──────┬──────┬──────┐

↓ ↓ ↓ ↓

Agent_01 Agent_02 Agent_03 ... Agent_N(Staff)（Mgr）(Exec) （新员工）

3.2 注册流程（完整时序）

步骤	内容	产出
1. IM 注册（推荐）	新员工扫码或点击链接 → lark-cli config init --new 自动创建专属飞书 Bot（PersonalAgent 框架自动配置 bot 能力/权限/事件订阅）。IM 不可用时降级为 WebUI 前端填表	app_id + app_secret + 用户 open_id 自动绑定
2. 后端写入	API 校验 → 写 users 表 → 加密存储 bot 凭证（app_id, app_secret）→ 生成 tenant_id → 签发 user_token (JWT) → 分配 role	用户记录 + bot 绑定 + tenant_id + user_token
3. 创建实例	根据 role 选 Agent 模板 → 注入 per-user bot 凭证（app_id, app_secret, tenant_id, user_token, gateway_url）→ docker run	Agent 实例容器（持有该员工专属 bot 凭证）
4. Gateway 注册	实例启动后向 Gateway 注册：POST /gateway/register {tenant_id, bot_id, agent_type, ws_url}	路由映射：bot_id → employee_id → tenant_id → ws_url
5. Bot 通知	员工的专属 Bot 发送飞书消息：「你的数字分身已就绪，开始对话」+ 初始配置引导	员工可开始使用

3.3 新员工拿到的参数

参数	来源	用途	持有方
tenant_id	后端自动生成	标识 Agent 实例，Gateway 路由用	Agent 实例 + Gateway
user_token	后端签发 JWT	调数据库 API 的鉴权	员工（前端）+ Agent
feishu_app_id	lark-cli 自动创建（per-user）	飞书消息收发	该员工的 Agent 实例
feishu_app_secret	lark-cli 自动创建，加密存库	飞书 API 鉴权	该员工的 Agent 实例
gateway_url	系统配置	WebSocket 连接 Gateway	Agent 实例
database_api_url	系统配置	调 CRUD API（通过 Skill）	Agent 实例

关键设计：每个员工拥有独立的飞书 Bot，app_id/app_secret 由 lark-cli 在注册时自动创建并加密入库。员工只需扫码即可完成 Bot 创建，无需知道任何技术参数。

3.4 Gateway 路由实现

3.4.1 飞书侧配置（自动完成，无需手动）

每个员工的 Bot 通过 lark-cli config init --new 创建时，PersonalAgent 框架自动完成以下配置，零手动控制台操作：

- Bot 能力：自动启用（mobile_default_ability: "bot", pc_default_ability: "bot"）
- 事件订阅：自动配置 WebSocket 长连接（callback_type: "websocket"）
- 权限范围：自动授权 32 个 scope（含 im:message:send_as_bot, im:message.p2p_msg:readonly 等）
- 应用状态：自动上线（status: 1）

Gateway 侧仍需配置统一的 Webhook 回调地址：https://gateway.automage.cn/feishu/webhook（各 Bot 的事件通过 Gateway 统一分发）

3.4.2 路由逻辑（伪代码）

```
# 飞书消息到达 Gateway
@app.post("/feishu/webhook")
```

```
async def handle_feishu_message(event: dict):
    sender_id = event["sender"]["sender_id"]["open_id"]
    receiver_id = event["message"]["chat_id"]

    # 查找 sender 对应的 Agent 实例
    tenant_id = db.get_tenant_by_open_id(sender_id)
    if not tenant_id:
        return "未注册，请先完成注册"

    # 路由到对应 Agent 实例
    ws_url = gateway.get_instance_url(tenant_id)
    await send_to_agent(ws_url, {
        "tenant_id": tenant_id,
        "sender_id": sender_id,
        "message": event["message"]["content"]
    })
    return "ok"

# Agent 实例启动后向 Gateway 注册
@app.on("startup")
async def register_to_gateway():
    config = load_config()
    await requests.post(
        f"{config['gateway_url']}/register",
        json={
            "tenant_id": config["tenant_id"],
            "agent_type": config["role"],  # staff/manager/executive
            "ws_url": f"ws://0.0.0.0:8080"
        }
    )
```

3.5 权限隔离机制

层级	机制	说明
IM 层（新增）	per-bot 天然隔离	每个员工独立 Bot（独立 app_id/app_secret），飞书平台侧天然隔离，单 Bot 故障不影响他人
数据层	tenant_id 隔离	

		所有数据库操作带 tenant_id 过滤，Skill 自动注入
通信层	Gateway 路由	消息按 bot_id → employee_id → tenant_id 路由，不会串
鉴权层	user_token (JWT)	调 API 时携带，后端校验 token 和 tenant_id 匹配

四、与里程碑二的对应关系

里程碑二任务	对应注册流程环节	负责人
数据库上线 + CRUD API	步骤 2：写入 users 表	熊锦文
数据库 Skill	Agent 实例启动后加载，用 user_token 调 API	熊锦文 + 胡文涛
三级 Agent 模板库	步骤 3：根据 role 选模板	胡文涛
Landing Page	注册页本身就是 Landing Page 的入口	张润诚 + 杨卓
多实例读写验证（5.8）	步骤 4：Gateway 路由验证	熊锦文

五、总结

- 注册时机：IM 优先（扫码创建 Bot，~30s），WebUI 作降级方案。先写库再下实例
- Gateway 接入：每个员工独立 Bot（免审权限 + lark-cli 自动创建），Gateway 做 bot_id → tenant_id 路由
- 飞书配置：PersonalAgent 框架一步到位（bot 能力/事件订阅/32 个权限/自动上线），零手动控制台操作
- 权限隔离：IM 层（per-bot 天然隔离）+ 数据层（tenant_id）+ 通信层（Gateway 路由）+ 鉴权层（JWT）
- 员工体验：扫码 → 自动创建专属 Bot → 系统完成后续配置 → Bot 发送欢迎消息
- Skill 角色：留给 Agent 日常读写用，不参与注册流程